



Robotics & Computer Vision

Week 4 — AI Vision: Object Detection for Robotics

YOLO · SSD · Bounding Box · Real-time Vision

Week 4 Learning Goals

1

Understand Object Detection

Learn what the object detection problem is and why it matters in computer vision and robotics.

2

Classification vs. Detection

Clearly distinguish between image classification, localization, and full object detection.

3

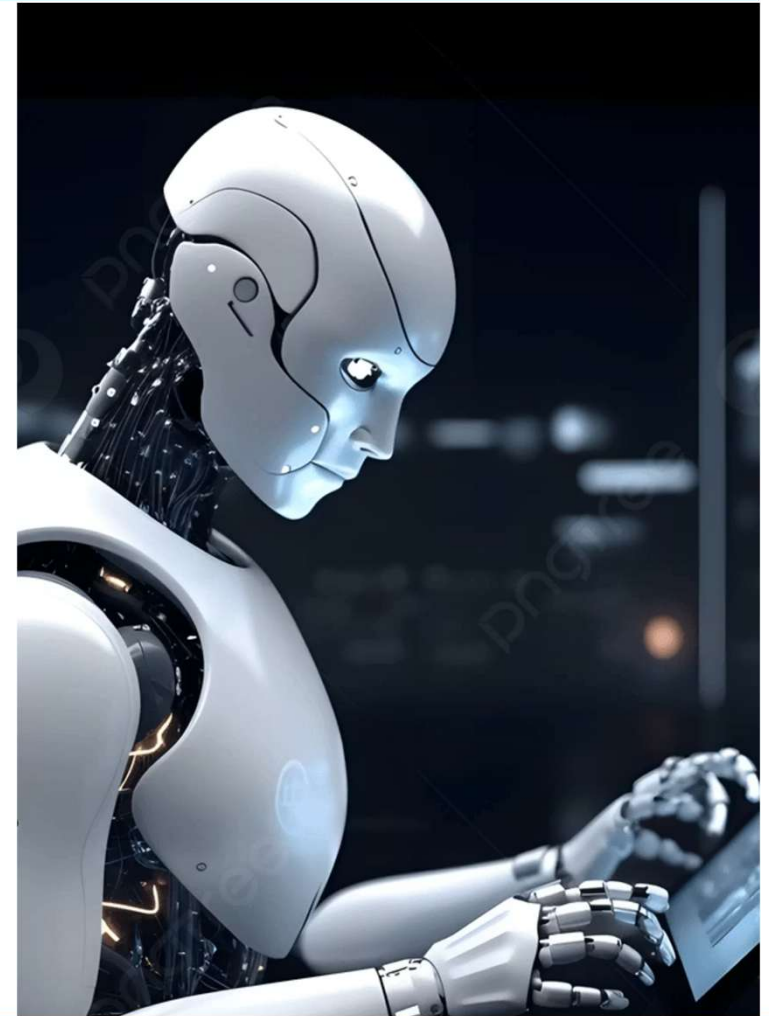
Bounding Box Essentials

Understand bounding box format, confidence score, and class label as detector outputs.

4

Robot Control Input

Extract the center of a detected box to use as a pixel target for robot navigation or tracking.



Week 4 in the Course Roadmap

RCV Roadmap

Week 4 sits at the heart of the AI Vision module. This week covers object detection, deploying YOLO/SSD, and extracting bounding boxes in real time — a key milestone in the Robotics & Computer Vision curriculum.

Practical Focus

The hands-on goal is to program a pipeline that reads a live video stream, runs a detector, and retrieves the center coordinates of a detected person or obstacle. This output feeds directly into robot control logic developed in later weeks.

Building on Week 3: From Classical CV to AI

1

Week 3 — Rule-Based Vision

Used manually crafted rules: **color thresholds**, **contour detection**, and **morphological operations** to **locate objects**.

2

Week 4 — AI-Powered Detection

Uses a **trained neural network** to **recognize arbitrary object categories**, far more robust to varied lighting and backgrounds.

3

Same Robot Goal

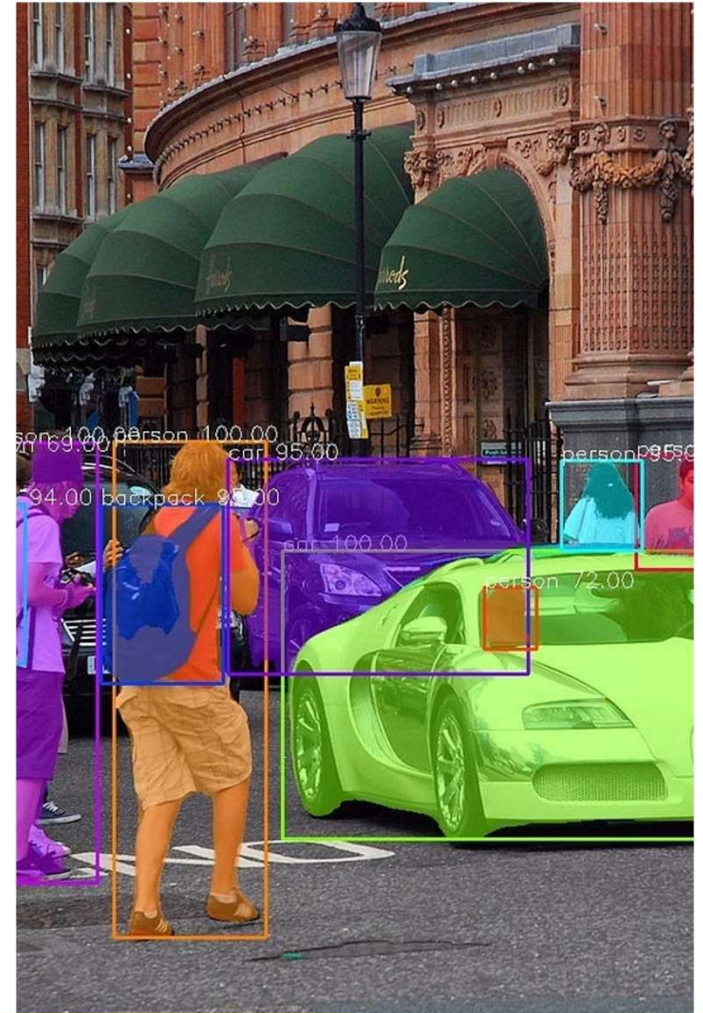
The output is still a pixel coordinate and a bounding region — the robot still needs a target pixel to follow or approach.

The transition from Week 3 to Week 4 mirrors the evolution of real robotics software: rule-based approaches are fast to prototype, but AI models generalize far better to the real world.

PART 1

What Is Object Detection?

Object detection is the computer vision task of **identifying what objects are present in an image and exactly where they are located**. The model answers two questions simultaneously: *What is it?* and *Where is it?*



Three Levels of Vision Tasks

Classification

What is in the image?

Returns a single label for the entire image. No location information.

Example: "This is a cat."

Localization

Where is the main object?

Returns one bounding box around the primary object in the image. Assumes only one object of interest.

Detection

What objects and where are all of them?

Returns multiple boxes with class labels and confidence scores. Handles many objects simultaneously.

For robotics, **detection** is the most useful task because real environments contain multiple objects of different types at the same time.

What Does a Detector Output?



Class Label

The name of the detected object category, e.g., *person*, *bottle*, *chair*. Comes from the set of classes the model was trained on.



Confidence Score

A value between 0 and 1 indicating how certain the model is about its prediction. Higher score = more reliable detection.



Bounding Box

The rectangular region surrounding the detected object, defined by corner or center coordinates within the image frame.



Multiple Results per Frame

A single image can yield many detections. Each detection is an independent (label, confidence, box) triple.



Why Does a Robot Need Detection?

→ Beyond Color Rules

A robot working in the real world needs to recognize *types* of objects, not just colors. Detection provides semantic understanding — knowing something is a "person" vs. a "box."

→ Handles Complex Environments

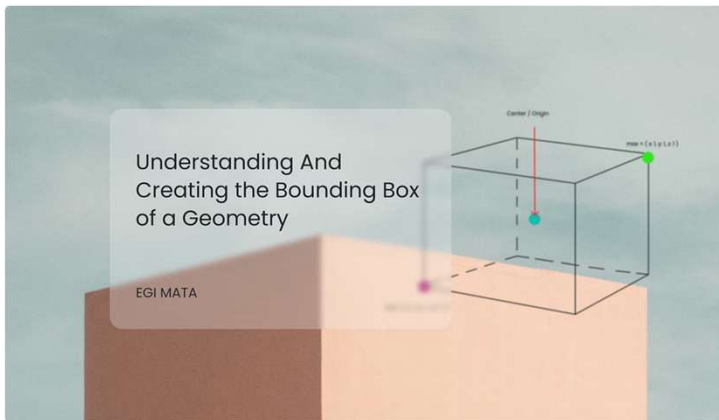
Real scenes have changing lighting, cluttered backgrounds, and shadows that break simple color-based rules. Trained models are far more robust.

→ Versatile Applications

Detection enables robots to find people, avoid obstacles, pick up bottles or boxes, and identify defective products on a production line.

→ Foundation for Advanced Skills

Detection is the first step toward object tracking, grasping, and intelligent manipulation — the core of modern robotics.



PART 2

Bounding Box

From the bounding region to a robot control target — understanding box geometry is the bridge between AI output and robot action.

How Is a Bounding Box Represented?

Format 1: Corner Coordinates

Use (x_1, y_1, x_2, y_2) — top-left and bottom-right corners of the rectangle. This is the most direct representation and is used by many frameworks including YOLO's `xyxy` format.

Format 2: Center + Size

Use (x, y, w, h) — center point plus width and height. Common in annotation formats and training pipelines. Both formats describe the same rectangle; converting between them is straightforward.

All coordinates are in the image pixel coordinate system, with origin at the top-left corner. The better the box fits the object, the better the localization quality.

Center of a Bounding Box

□ Center Extraction Formula

$$u = (x_1 + x_2) / 2$$

$$v = (y_1 + y_2) / 2$$

The center point (u, v) is the most practical way to convert an AI detection result into a pixel target. Rather than passing the full box to the robot controller, we give it a single point — the center of the detected object.

This pixel coordinate is then used by the robot to: **track** a moving person, **align** with a target object, or **navigate** toward or away from an obstacle. It is the critical link between the vision pipeline and the motion controller.

Understanding Confidence Score

What It Means

Confidence is the model's relative certainty about a predicted detection. It reflects both the probability that an object exists in that region and that it belongs to the predicted class.

Setting a Threshold

Common thresholds are **0.25** (lenient) or **0.5** (strict). Only detections above this threshold are kept. The right value depends on your application.

Low Threshold Trade-off

Detects more objects but introduces **false positives** — the robot might react to noise or irrelevant regions. Useful when missing an object is costly.

High Threshold Trade-off

Produces cleaner results but may **miss real objects**. Useful when false alarms are disruptive, but risks failure to detect obstacles.



YOLO and SSD

Two of the most widely used single-stage detector families — ideal for an introductory robotics course focused on real-time performance.

What Is YOLO?

You Only Look Once

YOLO is a one-stage detector designed from the ground up for **speed**. It treats detection as a single regression problem: the network sees the full image once and simultaneously predicts all bounding boxes and class probabilities.

Why YOLO for Robotics?

- Single forward pass = very fast inference
- Runs in real time on a webcam or embedded GPU
- Strong ecosystem: YOLOv5, YOLOv8, and beyond
- Easy to use via the `ultralytics` Python library
- Pre-trained on COCO (80 common object classes)

What Is SSD?

Single Shot Detector

Like YOLO, SSD performs detection in a single forward pass through the network — no separate proposal stage required.

Multi-Scale Feature Maps

SSD's key innovation is predicting boxes from **multiple feature map layers** of different spatial resolutions, making it naturally good at detecting objects of various sizes in the same image.

Educational Value

SSD is a classic reference architecture for explaining **anchor boxes** and **multi-scale detection** — core concepts for any student learning detector design.

YOLO vs. SSD: How Are They Different?

YOLO

- ✓ Simpler to use for demos
- ✓ Very high speed, great for webcam labs
- ✓ Large community, many pretrained weights
- ✓ Easy integration via **ultralytics**

Best choice for **robotics prototyping**

SSD

- ✓ Good for teaching detector architecture
- ✓ Illustrates anchor boxes clearly
- ✓ Multi-scale feature maps are explicit
- ✓ Classic benchmark reference model

Best choice for **understanding concepts**

For this course, we primarily use YOLO in labs due to its ease of deployment. SSD concepts are introduced to deepen architectural understanding.

One-Stage vs. Two-Stage Detectors

One-Stage Detectors

Examples: YOLO, SSD

Predict boxes and classes in a single network pass. Much faster — suitable for real-time robotics applications where latency matters.

Two-Stage Detectors

Example: Faster R-CNN

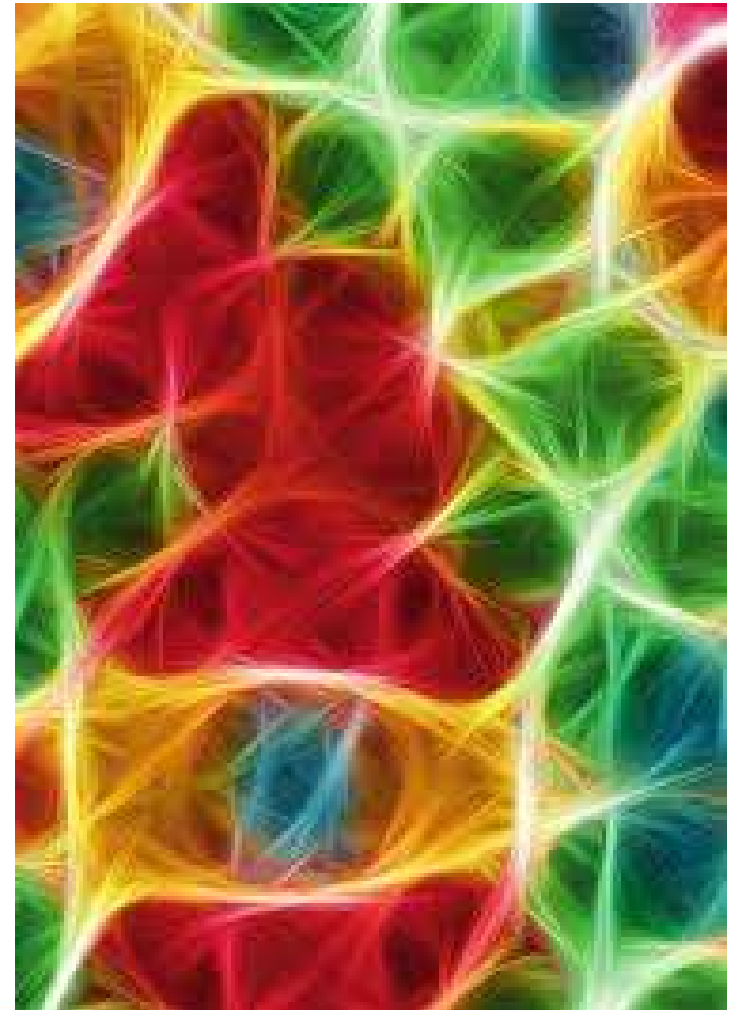
First generate region proposals, then classify each region. Historically stronger on accuracy benchmarks, but slower — less ideal for live robot control.

- ❏ For robotics applications where the robot must react in real time, **one-stage detectors are the practical default**. Two-stage detectors are worth knowing for historical context and accuracy-critical offline tasks.

PART 4

Deep Learning Foundations

The minimum background needed to understand how a modern detector actually works — no need to memorize every equation, but the intuition matters.



How Does a CNN Process an Image?

01

Convolution Learns Filters

Instead of manually designing edge or texture detectors, the network **learns optimal filters from data** during training. Each filter responds to a specific low-level pattern.

03

Feature Maps Carry Spatial Info

Each layer produces a **feature map** — a spatial representation encoding what patterns exist where in the image.

02

Deeper Layers = More Abstract

Early layers detect edges and colors. Middle layers detect textures and shapes. Deep layers detect object parts and eventually whole objects.

04

Detector Uses These Features

The detection head reads these feature maps and predicts bounding box coordinates and class labels for each detected region.

Detector Architecture: Backbone, Neck, Head



Backbone

Extracts rich visual features from the input image. Common backbones include ResNet, CSPDarknet (used in YOLOv5/v8), and EfficientNet. This is the core feature extractor.



Neck

Aggregates and combines features from multiple scales (e.g., FPN — Feature Pyramid Network). This enables the detector to handle both large and small objects effectively.



Head

The final prediction layer. Outputs class probabilities and bounding box coordinates for each detection. This is what produces the results you use in your robot pipeline.

Why Is Detection Harder Than Classification?

Location Required

Not just "what" but "where."
The model must predict precise box coordinates, which is a continuous regression problem on top of classification.

Multiple Objects

A scene may contain 10 people, 3 bottles, and 2 chairs simultaneously. The model must handle a variable number of detections per image.

Scale Variation

Objects appear at many sizes and viewing angles. A person 2 meters away looks very different from one 10 meters away. The model must generalize across all of these.

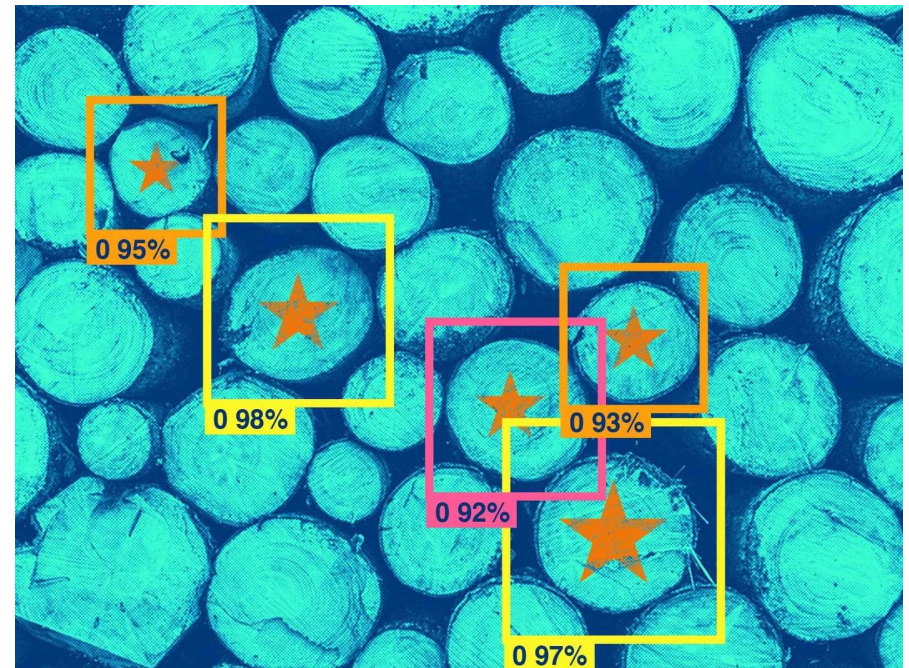
Complex Backgrounds

Cluttered backgrounds, partial occlusions, and shadows all make it harder to isolate and identify objects. This is where trained models significantly outperform rule-based approaches.

PART 5

Training and Data

A detector is only as good as the data it learns from. Understanding data requirements helps you build reliable models for real robotics applications.



Datasets for Object Detection

→ Bounding Box Annotations Required

Unlike image classification, every training image must have one or more bounding boxes drawn around each object of interest, with precise pixel coordinates.

→ Each Box Needs a Class Label

Every bounding box must be labeled with the object category it encloses. Consistent labeling is critical — ambiguous labels degrade model performance.

→ Diversity Is Essential

Data must include varied viewpoints, lighting conditions (bright, dim, backlit), and backgrounds. Diversity is what allows the model to generalize beyond the training environment.

→ Lack of Diversity = Weak Model

A model trained only on studio photos will fail in a real warehouse or outdoor setting. Data quality and diversity matter more than data quantity alone.

Train / Validation / Test Split

Train

Used to **update model weights** during gradient descent. The model sees this data repeatedly across many epochs. Typically 70–80% of the dataset.

Validation

Used to **monitor performance during training** and tune hyperparameters such as learning rate and confidence threshold. Helps detect overfitting early. ~10–15% of data.

Test

Used **only for final evaluation** — after training is completely done. Never touched during training or hyperparameter tuning to ensure an unbiased assessment. ~10–15% of data.

📌 A common mistake is to tune on the test set. This gives overly optimistic results and a model that fails in real deployment. Keep the test set truly held out.

Data Augmentation

What Augmentation Does

Augmentation artificially expands the dataset by applying transformations: **horizontal flip, random crop, resize, color jitter, brightness change, rotation**. This teaches the model to be robust to real-world variation.

It is one of the most cost-effective ways to improve a detector without collecting new data.

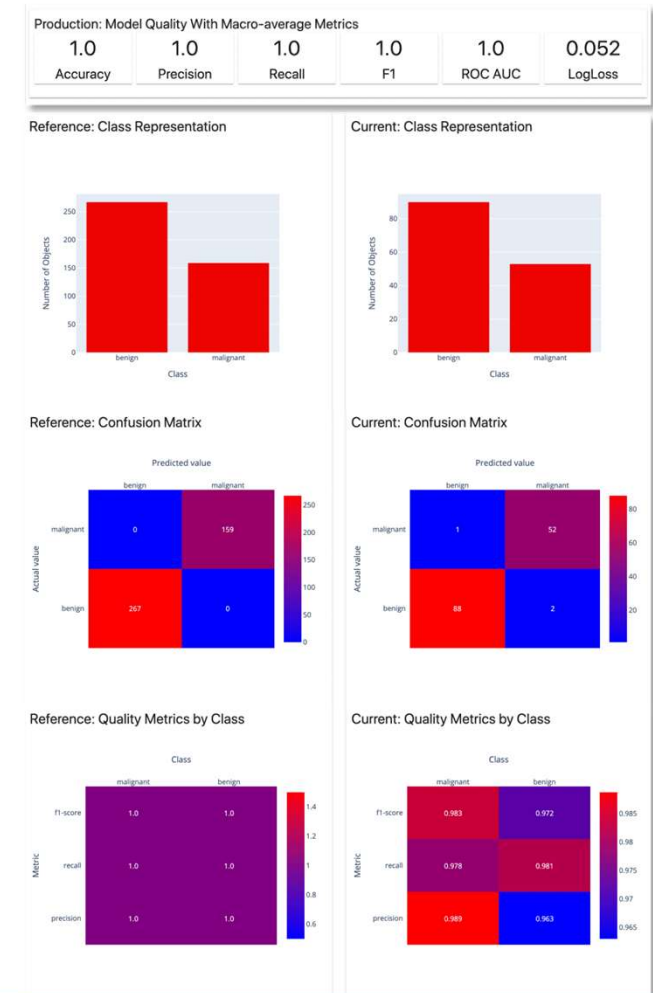
Important Rules

- Geometric transforms (flip, crop) **must update the bounding box coordinates** to stay aligned with the transformed image
- Color-only transforms (brightness, hue) do not change box coordinates
- Avoid augmentations that distort object meaning (e.g., extreme vertical flip)
- Overly aggressive augmentation can confuse the model

PART 6

Evaluating the Model

Visually good-looking boxes are not enough. Understanding evaluation metrics is essential to know whether your detector is truly ready for deployment.



What Is IoU?

Intersection over Union

IoU measures the overlap between the predicted bounding box and the ground truth box. It is defined as:

$$\text{IoU} = \text{Area of Overlap} \div \text{Area of Union}$$

A value of **1.0** means perfect overlap. A value of **0** means no overlap at all.

How to Interpret IoU

- $\text{IoU} \geq 0.5$ is typically considered a correct detection
- $\text{IoU} \geq 0.75$ indicates a tight, high-quality box
- $\text{IoU} < 0.5$ means the box is too far from the real object position

IoU is used to determine whether a detection counts as a True Positive or a False Positive when computing precision and recall.

Precision and Recall

Precision

Of all detections made, how many were correct?

High precision = few false alarms. The model only predicts when it is confident. Important when false positives cause disruptive robot behavior.

Recall

Of all real objects, how many were found?

High recall = few missed detections. The model catches everything. Critical in robotics: missing a person or obstacle could be dangerous.

The Trade-off

Increasing the confidence threshold improves precision but reduces recall. The right balance depends on the application's safety requirements and tolerance for errors.

📌 In robotics safety scenarios, **recall is often prioritized** — it is better to have a false alarm than to miss a real obstacle entirely.

mAP — An Intuitive Understanding

Mean Average Precision

mAP summarizes detection quality across **all object classes** by averaging the precision-recall curve for each class. It is the standard benchmark metric for comparing detectors.

What the Number Means

A higher mAP generally indicates a better detector on that benchmark dataset. Common values: mAP@50 (IoU threshold 0.5) and mAP@50:95 (averaged over multiple IoU thresholds — stricter).

Week 4 Scope

You do not need to derive the mAP formula right now. The key takeaway: **higher mAP = better overall model on the benchmark**. Use it as a guide when comparing pretrained models or checking your own training results.

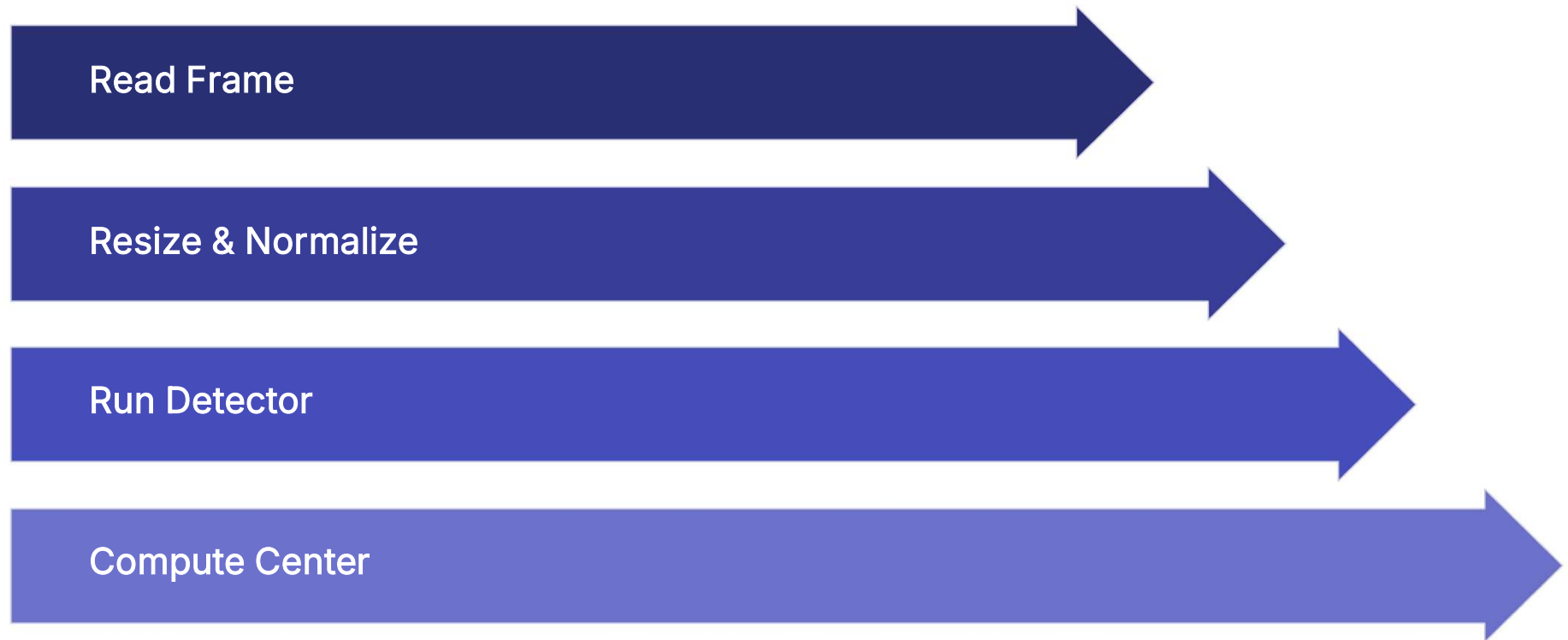
PART 7

Deployment Pipeline

From camera feed to bounding boxes and target pixel coordinates — putting it all together in a working system.



Basic Inference Pipeline



This four-step loop runs on every video frame. In a real robot system, the output of Step 4 — the center coordinates (u, v) — is passed directly to the motor or servo controller to drive the robot's response.

Non-Maximum Suppression (NMS)

The Problem

A detector can predict **many overlapping boxes** for the same object because it checks multiple anchor positions and scales. Without NMS, a single person might generate 10–20 redundant boxes.

The Solution

NMS keeps only the **highest-confidence box** and removes all nearby overlapping boxes above an IoU threshold.

- Sort all boxes by confidence score
- Keep the top box, suppress overlapping ones
- Repeat until no overlapping boxes remain
- Result: one clean box per object

NMS is automatically applied by frameworks like `ultralytics`, but it is important to understand what is happening under the hood.

FPS and Latency in Real-Time Robotics



Why Speed Matters

A robot needs to react to its environment in real time. Low FPS means the robot is acting on stale visual information — it may still be turning toward a position the person left 500ms ago.



Resolution Trade-off

Higher input resolution gives better detection accuracy, especially for small objects, but dramatically increases inference time. Choose the minimum resolution that meets your detection requirements.



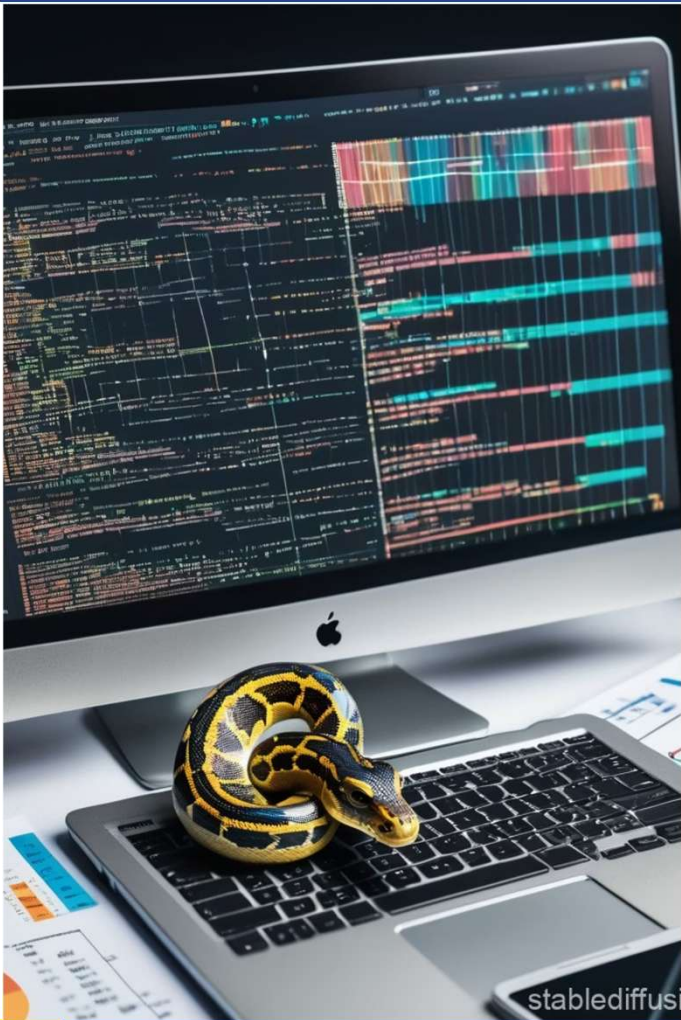
Hardware Matters

A GPU or edge AI chip (e.g., NVIDIA Jetson) can run YOLO at 30+ FPS. A CPU-only system may only achieve 5–10 FPS. Hardware selection is part of system design.



Model Size vs. Task Need

Do not default to the largest model. YOLOv8n (nano) is often sufficient for robotics demos and runs much faster than YOLOv8x. Match model size to your actual performance requirements.



PART 8

Python Hands-On Practice

We use a pre-trained YOLO model via the `ultralytics` library — so you can focus on the robotics application rather than re-implementing the neural network from scratch.

Lab 1: Setting Up the Environment

```
# Install required packages
pip install ultralytics opencv-python

# Import in your Python script
from ultralytics import YOLO
import cv2
```

ultralytics

The official YOLO Python library. Provides a clean API for loading models, running inference, and parsing results.

opencv-python

Used for reading images and video streams, drawing boxes and annotations, and displaying frames in a window.

Lab 2: Load a YOLO Model and Run Inference

```
# Load the pre-trained YOLOv8 nano model
model = YOLO("yolov8n.pt") # downloads automatically on first run

# Read a still image
img = cv2.imread("scene.jpg")

# Run detection
results = model(img)
```

- ❏ **yolov8n.pt** is the "nano" variant — the smallest and fastest YOLOv8 model. It is pre-trained on COCO (80 classes including person, bottle, chair, etc.). The model file downloads automatically the first time you run this code.

Lab 3: Reading Boxes and Classes

```
for r in results:
    for box in r.bboxes:
        cls_id = int(box.cls[0])          # class index (e.g., 0 = person)
        conf   = float(box.conf[0])      # confidence score (e.g., 0.87)
        x1, y1, x2, y2 = box.xyxy[0]    # box corners in pixels

        label = model.names[cls_id]      # human-readable class name
        print(f"{label}: {conf:.2f} box=[{x1:.0f},{y1:.0f},{x2:.0f},{y2:.0f}]")
```

For each detected object, `r.bboxes` gives you the class index, confidence, and box coordinates. Use `model.names[cls_id]` to convert the index to a readable label like **"person"** or **"bottle"**.

Lab 4: Computing the Box Center

```
# Compute center of the bounding box
u = int((x1 + x2) / 2)
v = int((y1 + y2) / 2)

# Draw a red dot at the center on the image
cv2.circle(img, (u, v), 5, (0, 0, 255), -1)

# Optionally draw the bounding box itself
cv2.rectangle(img, (int(x1), int(y1)), (int(x2), int(y2)), (0, 255, 0), 2)
```

- ❏ The coordinates **(u, v)** are the pixel target for your robot controller. Pass these values to your motor control code to make the robot track, approach, or avoid the detected object. This is the core connection between AI vision and robot action.

Lab 5: Real-Time Webcam Detection

```
cap = cv2.VideoCapture(0) # 0 = default webcam

while True:
    ret, frame = cap.read()
    if not ret:
        break

    results = model(frame)

    for r in results:
        for box in r.bboxes:
            cls_id = int(box.cls[0])
            conf = float(box.conf[0])
            x1, y1, x2, y2 = box.xyxy[0]
            u = int((x1 + x2) / 2)
            v = int((y1 + y2) / 2)
            # Draw box, label, confidence and center point
            cv2.circle(frame, (u, v), 6, (0, 0, 255), -1)

    cv2.imshow("YOLO Detection", frame)
    if cv2.waitKey(1) & 0xFF == ord('q'):
        break

cap.release()
cv2.destroyAllWindows()
```

WEEK 4 WRAP-UP

From AI Detection to Robot Action

Week 4 takes you all the way from the concept of object detection to a working real-time pipeline. You now have the tools to make a robot **see, identify, and respond** to objects in the world.

Assignment

Use YOLOv8 to detect a **person** or a **water bottle** from your webcam. Compute and display the bounding box center **(u, v)** on each frame. Print the **confidence score** alongside the class label. Bonus: filter out detections below confidence 0.4.

Next Week Preview

Week 5 will use these pixel coordinates to drive actual robot movement — closing the loop from vision to motor control. The center point you compute this week becomes the error signal for a control algorithm next week.